The Feature Ledger

VERSION 2 · SIMPLIFIED API

Everything feature-ledger does today, in plain language. Each entry is one capability you could ask for by name, sized so the biggest pieces stand out.

What's changed since the last edition (September 8, 2026): every entry below is highlighted to show what's different. A green stripe marks a brand-new feature; a blue stripe marks an existing feature that was materially reworked, with its own **What changed** note. Anything without a stripe hasn't changed.

New feature Updated feature

THE LEDGER AT A GLANCE

25 features across **4** areas

EVERY ONE OF THOSE 25, BY SIZE

9 BIG	10 MEDIUM	6 SMALL

BIG — A capability the product is chosen for

MEDIUM — A capability in its own right, inside a bigger one

SMALL — A capability narrow enough to describe in a line

SINCE THE LAST EDITION, SEPTEMBER 8, 2026

5 NEW **8** UPDATED **12** UNCHANGED

CONTENTS

01 Setting up a ledger 6

02 Recording what the product does 11

— No longer available as of this edition 2

03 Reading the ledger 3

04 The documents it produces 5

— Also since last time 5

01 Setting up a ledger

6 FEATURES · 2 UPDATED

BIG

UPDATED

Creating a ledger in an existing project

The command ledger init creates a .ledger directory holding the whole corpus: the product name, the accent colour and the style guide you chose, and version 1 open as the release in progress. Everything the tool needs sits in that one directory, so a Python, Go or Rails repo looks identical to it. Setup also settles where the generated documents go. The Markdown docs are build output and are added to .gitignore. The PDF editions for the client are build output too, unless you run ledger init --commit-pdfs, which keeps them in docs/client so every document handed over stays in the repo.

WHAT CHANGED

- Setup gitignored the client PDFs along with the Markdown docs, so the repo kept none of the editions handed over. A team that hands a printed edition to a client at each release wants the repo to hold what was handed over, so where the editions land is now a choice made once at setup.

BIG

The tone every entry is written in

A ledger is written in one tone, chosen from four style guides: Google developer documentation, GOV.UK, Federal Plain Language, and ASD-STE100 Simplified Technical English. The command ledger style prints that tone as a sample paragraph of product prose and three feature descriptions written in it, which is the thing a writer or an agent matches. Running ledger style set moves the project to another guide, or to a tone you write as prose yourself in .ledger/STYLE.md.

MEDIUM

Wiring into a coding agent

Setup appends a short stanza to whichever agent instruction files the repo already has — CLAUDE.md, AGENTS.md, GEMINI.md, .cursor/rules, .github/copilot-instructions.md or .windsurfrules — and creates AGENTS.md when there are none. The stanza is a pointer rather than a copy: it names six commands, the tone the project writes in, the line between recording a change and rewording an entry, and the rule that a feature change is not finished until the ledger records it. The --agents flag chooses the files, and changing the tone rewrites the stanza in every one of them.

MEDIUM

UPDATED

Baseline survey of an existing codebase

The command ledger bootstrap prints a brief you paste into a coding agent, which then reads the codebase and records what it already does as version 1. The brief carries the entry bar, the project's tone, and five steps: list the capabilities, decide the areas, write the entries, cut the baseline from the date of the first commit, and hand back the entries it was least sure of. The brief also records the survey as the first full sweep of the ledger audit, so the audit counts unrecorded work from the baseline onward. The agent runs it start to finish without stopping to ask you anything.

WHAT CHANGED

- The survey counts as the first full sweep of the ledger audit; see “The ledger audit”. The brief records it just before the baseline is cut, the one moment when what the ledger holds and what the codebase does are known to agree. Pass --no-audit for a survey that is deliberately partial.

SMALL

Moving an existing ledger to a new tone

The command ledger style rewrite prints a brief for rewriting a ledger that is already written into the tone the project has moved to. The agent works one area at a time and replaces the words in place, so nothing lands on the release and no entry is marked as changed.

SMALL

The authoring rules

The command ledger rules prints the full rules for writing the ledger: what each field means, where the bar for an entry sits, how to record a change or a removal, and which parts of the corpus you edit by hand instead. The project's tone is printed inside it, and a RULES.md in the ledger directory replaces the packaged text with your own house conventions.

02 Recording what the product does

11 FEATURES · 5 NEW · 2 UPDATED

BIG

Recording a capability

The command ledger add records one capability, with the audience it matters to, its size, its name and a present-tense description of what the product does. The id you give it is lower-case kebab-case and never changes, so it carries the feature's history through every later rename. An entry filed under an area or a sub-section that doesn't exist is refused as you write it, and the message lists the ones that do.

BIG

UPDATED

Recording a change to a feature

The command ledger update replaces a feature's description with its full current-state text, and takes a list of change bullets alongside it. Each bullet says what the feature was in the last edition and why it moved, which is the part a reader cannot get from the description. A feature that existed before this release cannot be updated without one, so an edition never carries a changed entry with no reason attached. A feature that is new in this release takes no change bullets, because the client has never seen an earlier version of it. When a feature changes a second time in one release, the command reminds you that its bullets have to be true of the last edition, not of the state your previous update left.

WHAT CHANGED

- A change bullet could be added to a feature that was new in the same release, and a second change in one release got no warning. Each bullet then described the state just before its own update, which the client never saw, so an edition could explain a fix to something that was new in the same release. The command now refuses bullets on a new feature and reminds you which edition a second change is measured against.

BIG

NEW

Editing the ledger by hand

The corpus is JSON in the repository, one small file per feature, and its prose is yours to rewrite. You open the file and reword a name, a description, a dev note or the text of a change bullet, add the sentence a coding agent had no way of knowing, or cut a detail the client is better off not reading. Nothing is recorded against a release and nothing is marked as changed, because the product did not move, so the improved wording reads as though it had always been there, including in any earlier edition you reprint. The command ledger status then checks every file, so a description that went missing comes back as a message rather than as a broken build. A change to what the product does goes through ledger add, update or remove, which place it on the right release and refuse a change that never says why it happened.

BIG

NEW

The ledger audit

The command ledger audit finds capabilities that shipped without the ledger recording them, such as work done without a coding agent that read the project's instructions. It prints a brief for a coding agent: the range since the last audit, the changed paths grouped into batches, and the commit subjects behind them. An audit covers everything since the last one, or the whole codebase with --full, which is the only way to find a capability the baseline survey missed. The agent records what it finds and then stamps the commit it audited, so the tool knows how much of the repository has not been examined yet. A release with unaudited work behind it cannot be cut, and the refusal says how many commits and files are outstanding. A project that started with ledger bootstrap is already stamped, because the baseline survey counts as a full sweep.

BIG

NEW

Reviewing a release before it goes out

The command ledger review prints a brief for a coding agent to read the open release the way the client will: as one document, against the edition they already have. The brief lists what arrived in the release, every client-facing entry in it by area, and the places the drafts probably show, such as change bullets that name something the previous edition never mentioned. The agent tests every bullet against the previous edition, drops or merges the ones whose "before" never shipped, removes change notes from new entries, and makes names, numbers and cross-references agree. It edits wording only and never invents a reason, and it ends with a list of questions for you. ledger review complete records what was reviewed, and ledger status reports any client-facing entry that changed after that.

MEDIUM

Grouping features into areas

Areas group the entries, and the order of the list is the reading order of every document the ledger produces. The command ledger categories add places one, before or after any other; rename and remove move the structure without touching the entries underneath it. A ledger with no areas puts everything in one called Features and prints it without a heading, so a small product reads as a list of what it does rather than as a chapter of one.

MEDIUM

NEW

Redrafting the ledger

The command ledger redraft start sets the corpus aside in .ledger/archive/ and opens the next version on an empty one, so you can write the ledger again from a fresh survey. Version numbers carry on, every edition already printed keeps its number and its file, and ledger build with a version number reprints an archived edition from its archive. The command ledger redraft check prints a brief for a coding agent to read every archived entry against the new corpus, put back anything the survey missed, and write a note for the client on how the arrangement changed. Until ledger redraft complete records that check, ledger build warns and ledger cut refuses, and ledger redraft on its own says what is left to do. The first edition after a redraft opens with a note that the ledger is reorganized, and marks nothing as new or updated. A redraft you do by hand, without ledger redraft, gets the same note when you set "redraft": true on its release in releases.json. That edition reuses the number of the edition before it, and its file name ends in _redraft, so Version 3 is followed by Version 3 (redraft) and then Version 4. Set on the first release, the note tells the client that the list replaces reports they received some other way.

MEDIUM

NEW

Handing a task to a coding agent

The commands that prepare work for a coding agent — bootstrap, audit, review, redraft check and style rewrite — print a prompt for you to paste. Add --copy to put it on the clipboard, or --run claude, codex or gemini to hand it to that agent and wait; any other command that reads a prompt on stdin works too. When the agent stops, you see its last message, whether it recorded what its task asks for, and the command that resumes its session. Nothing is handed to an agent unless you pass --run.

SMALL

Retiring a feature

The command ledger remove marks a feature as gone from this release onward, with the reason you give it. It appears once, under the heading No longer available as of this edition, and every earlier edition still shows the feature as it stood at the time.

SMALL

UPDATED

Recording a product-wide change

The command ledger note records something that runs across the whole product, such as a wording pass over many screens or a change in what a colour means. It prints once at the end of the edition, under the heading Also since last time, instead of forcing the same note onto a dozen unrelated entries.

WHAT CHANGED

- The command was ledger other-change. It is now ledger note, which names what it records; the old name still works and says what it is called now.

SMALL

Sub-sections inside an area

A sub-section is a heading between an area and a feature, for an area with a real part inside it. Each one carries a name and a one-or-two-sentence overview of that part, and you file a feature into it when you record or update the feature. When every feature in a sub-section arrives in the same release, the edition flags the sub-section once rather than printing a column of identical New marks.

03 Reading the ledger

3 FEATURES · 1 UPDATED

MEDIUM

Listing and reading entries

The command ledger list prints the whole corpus, one short line per feature: its size, its id, its name, and whether it is new or changed in the release in progress. It filters by area, audience, size, or by what has moved this cycle. The command ledger show prints the single entry you are about to edit, and prints every version it has been through when you ask for its history.

MEDIUM

UPDATED

Checking the ledger is sound

The command ledger status answers four questions at once: what the open release has collected, whether it is ready to cut, what the corpus holds, and how the setup resolves, from the derived palette to the browser that will print. The readiness answer is the same checklist ledger cut runs, with the command that fixes each open check, and anything that is valid but would still print badly comes out as a warning at the end. Running ledger on its own prints a short version: what the open release holds, the checklist, and the one command to run next. ledger status exits with an error when the corpus is broken, and with --quiet it stays silent when there is nothing to say, so a commit hook can run it on every commit.

WHAT CHANGED

- ledger status showed what the open release held and warned about what would print badly, but it did not say whether the release could be cut: you found out by trying, one refusal at a time. It now shows the checklist ledger cut runs (see “Cutting a release”), and ledger on its own says where things stand and what to run next, so nobody has to remember the order of audit, review and cut.

SMALL

Exporting the whole ledger

The command ledger export prints the entire corpus as a single JSON document: the release timeline, the areas, the sub-sections, the product-wide changes, and every feature with its full history. It writes to a file instead when you name one, for a backup, a migration, or handing the data to something that isn't this tool.

04 The documents it produces

5 FEATURES · 3 UPDATED

BIG

UPDATED

Cutting a release

The command ledger cut gives the open release its name and date, prints its PDF edition into the client folder, and opens the next release, where everything recorded from then on lands. Before it writes anything, it runs five checks: no redraft is waiting to be checked, no code has changed since the last audit, something is recorded, the release is reviewed, and the cut stands for a different moment from the last one. When a check is open, the cut writes nothing, lists every check with the command that fixes it, and --skip passes only the checks you name. Each cut records its commit, so two demos on one day stay two editions, and ledger releases shows the timeline with each release's commit.

WHAT CHANGED

- A release was a name and a date, and the date was doing work it could not do: two demos in one day are ordinary, and nothing said which of the two editions a reader was holding or which state of the code it came from. Every cut now records its commit. Sharing a date is still fine, but sharing a date and a commit is refused, because the two editions would stand for the same moment; --skip moment covers a deliberate re-issue.
- A release could be cut as soon as it had something recorded against it, and nothing checked that its entries read correctly as one release. A cut now needs the release reviewed first, unless it is the first edition, and it also waits for the ledger audit and for any redraft to be checked. See “Reviewing a release before it goes out”, “The ledger audit” and “Redrafting the ledger”.
- Cutting was ledger release cut followed by a separate ledger build, and a refused cut printed one line about the first problem it hit. That refusal was easy to miss: the build that followed printed the open release under the file name the cut edition would have had, so it looked as though the cut had worked. The command is now ledger cut, it prints the edition itself, and a refusal says plainly that the release was not cut and lists every open check at once.

BIG

UPDATED

The printed edition for the client

Each edition is a PDF: a front page carrying the masthead, the feature count, a bar breaking those features down by size, and a note of what changed since the last edition, then a contents list that links into the document, then every entry in reading order. ledger cut prints it into the client folder; ledger preview prints the open release as a draft, named with -draft and kept in the gitignored docs folder, so a draft is never mistaken for an edition. ledger build prints every cut edition that isn't on disk yet, and ledger build v3 reprints one with its entries as they stood that day, in their current wording. The fonts are embedded, so text copies cleanly out of the finished file, and display_version on a release in releases.json sets the number its edition prints.

WHAT CHANGED

- The contents list was a printed index: it named each area and how many features were in it, but reaching a section meant scrolling for it. Every line is now a link to the section it names, and the closing sections on what was withdrawn and what else changed are listed there too, because the edition is usually read on screen rather than on paper.
- Every edition printed the ledger's own version number, starting from 1, so a project whose client already held reports sent another way before the ledger started handed them a second Version 1. A release can now set the number its edition prints, and later editions carry on from it; the ledger's own numbering is unchanged.
- ledger build printed one PDF: the newest edition, or the open release as a draft once it had work in it, under the same file name and in the same folder the cut edition would use. A draft could be mistaken for the edition the client was handed. A draft is now named with -draft and kept out of the client folder, only ledger cut writes into it, and ledger build also prints any cut edition that isn't on disk.

MEDIUM

Three Markdown documents for the team

The command ledger build also writes three Markdown documents from the same corpus: FEATURES.md, the client-facing list; FEATURES_EXTENDED.md, the same list with implementation notes under each entry; and DEV_FEATURES.md, the entries only the team sees. All three describe the latest release, so they read as what the product does today. They are build output: the corpus is what you commit, and it carries the documentation's history on its own.

MEDIUM

UPDATED

New and changed highlighting

Every edition marks its own changes against the edition before it: a green stripe for a feature that is new, a blue stripe and a "What changed" note for one that was reworked, a grey stripe marked Already in place for one that was already part of the product and appears in the ledger for the first time, and no stripe for an entry that stands as it did. The front page counts how many of each. A first edition has nothing behind it to compare against, so it prints as a plain inventory with no stripes anywhere on it.

WHAT CHANGED

- There were two stripes, so a capability recorded late printed as new or updated. A capability that had been in the product for two years then read as this sprint's work, which is the one thing a client would take issue with. The grey stripe says what is actually true: the product did not move this cycle, the record did. Entries the ledger audit turns up are the usual case; see “The ledger audit”. It is a neutral grey rather than a fourth hue because green and blue are already spoken for, an accent in that band is refused for the same reason, and a fifth hue would leave a brand only reds, oranges, purples and browns to choose from.
- Whether an entry printed as new or updated depended on whether it had change bullets. A new feature that picked up a change bullet during the release printed as updated, with a "What changed" note about a version the client never saw. It is now new whenever the feature first appears in that release, and a new entry never prints a change note.

MEDIUM

Branding the documents for a client

One file, .ledger/brand.json, is the swap surface for a new client: a logo, one accent colour, and the two font families. The other four tints of the accent are derived from the hex you give it, and a ledger with no logo sets the product name as a serif wordmark on the front page. The accent cannot be green, blue or teal, because those hues already mean new and reworked in every edition, and ledger status warns when a brand picks one that collides.

— No longer available as of this edition

Rewording an entry — Rewording an entry is no longer a command. You edit the words in .ledger/features/<id>.json directly, which records nothing against a release and marks nothing as changed: the same result the command produced, without a CLI between a person and a sentence they want to fix. See “Editing the ledger by hand”.

Renaming a feature — Renaming a feature is no longer a command of its own. A badly chosen name is wording, and you edit it in .ledger/features/<id>.json; see “Editing the ledger by hand”. A name the product actually changed is recorded with ledger update --name, which carries the reason the reader is given. Every document still footnotes the previous name.

Also since last time

Changes that run across the whole product rather than belonging to any one feature above.

- The tool is published on npm as feature-ledger. You install it once with `npm install -g feature-ledger` and run `ledger` in any project, add it to a JavaScript project with `npm install --save-dev feature-ledger`, or run any command through `npx feature-ledger` without installing anything. Using it no longer means keeping a checkout of the tool itself.
- Every command that prints a prompt for a coding agent (ledger bootstrap, ledger style rewrite, ledger audit, ledger review and ledger redraft check) takes --print, which prints the prompt alone. You can pipe it straight to the clipboard, for example with ledger review --print | pbcopy. Warnings print on your screen rather than into the pipe. See “Handing a task to a coding agent” for the other ways to deliver a prompt.
- Adding --help or -h to any command prints that command's part of the help and runs nothing. For example, ledger cut --help never cuts a release.
- Every command refuses an option it does not take, and runs nothing. The error names the option, suggests the one you probably meant, and lists the options that the command takes. For example, ledger cut --dryrun stops with "did you mean --dry-run?" instead of cutting the release.
- The commands read as words you can guess: ledger cut, ledger releases and ledger subcategories replace ledger release cut, ledger release list and ledger subcategory. The old names still work and say what they are called now. A mistyped command is answered with the closest one, the exit code tells a mistake (2) from a check that said no (3), and ledger completion adds tab completion to zsh, bash and fish.

feature-ledger · The Feature Ledger · Version 2 · October 10, 2026. Prepared for the client; every entry describes behaviour that is built and running.